存储管理

刘斌

武汉大学计算机学院
binliu@whu.edu.cn

内容

- **数据组织**
  - 记录组织
  - 页面组织
  - 文件组织
- **空闲空间管理**
- **GaussDB案例**
- **缓存管理**

定长记录

定长记录

```
create table SC(
SID char(10),
CID char(6),
Degree smallint
);
```

SID	CID	Degree
2023011789	CS101	98
2023011689	CS101	83
2023011689	CS102	93



定长记录——删除

定长记录

create table SC(

SID char(10),

CID char(6),

Degree smallint

);

SID	CID	Degree
2023011789	CS101	98
2023011689	CS101	83
2023011689	CS102	93





问题：移动代价太大，修改索引代价大

定长记录——删除

定长记录

create table SC(

SID char(10),

CID char(6),

Degree smallint

);

SID	CID	Degree
2023011789	CS101	98
2023011689	CS101	83
2023011689	CS102	93

给每个记录增加记录头信息，记录该记录是否被删除，0代表没有，1代表删除。





空值处理

定长记录

create table SC(

SID char(10),

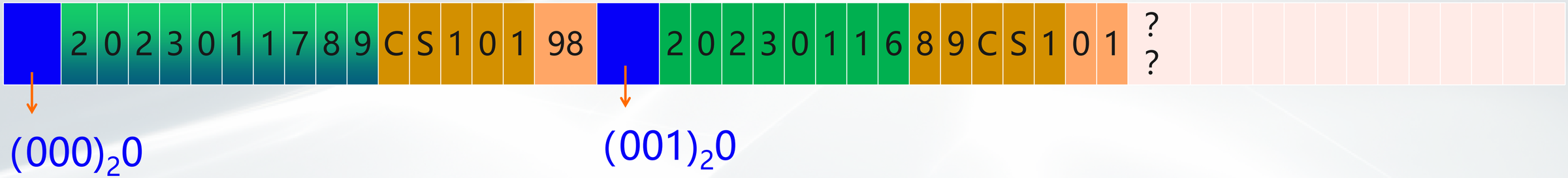
CID char(6),

Degree smallint

);

SID	CID	Degree
2023011789	CS101	98
2023011689	CS101	null

同样在记录头信息增加空值二进制指示位，N个字段N个位，1代表为空，0代表不为空



记录头信息（Head） **记录本身（Data）**

变长记录

```
create table Student(
SID char(10),
Name varchar(20),
GPA decimal,
Dept varchar(10)
);
```

SID	Name	GPA	Dept
2023011789	Wu Jun	3.9	计算机
2023011689	Li Bing	3.78	数学

2	0	2	3	0	1	1	7	8	9	W	u		J	u	n	\0	3.9	计	算	机	\0	2	0	2	3	0	1	1	7	8	9	L	I		B	i	n	g	\0	3.78	数	学	\0

先存定长字段，再存变长字段：起始地址+长度

	$(0000)_2$0	2023011789	3.9	(32,7)	(39,7)	Wu jun	计算机
字节	0	10	20	24	28	32	39

变长记录


记录头信息（Head）
记录本身（Data）
定长字段1
……
定长字段n
地址1：长度1
……
地址m：长度m
变长字段值1
……
变长字段值m

页面组织——定长记录

一个页面通常是4KB、8KB或者16KB，通常是2的整数幂。

- 页头
  - 记录数
  - 校验和
  - 上下页面指针
  - 记录删除位状态向量（每一个元组不需要删除位）
- 数据

记录数量、校验和、上下页面指针				记录删除状态位向量 (111111000000000000)	
记录1	记录2	记录3	记录4	记录5	记录6
记录7	记录8	记录9	记录10	记录11	记录12
记录13	记录14	记录15	记录16	记录17	记录18

因为是定长记录，可以直接计算出记录分配的地址空间！！！

页面组织——定长记录

定长记录

create table SC(

SID char(10),

CID char(6),

Degree smallint

);

SID	CID	Degree
2023011789	CS101	98
2023011689	CS101	83
2023011689	CS102	93

PageHeader

- **记录数量——2B**
- **校 验 和 ——2B**
- **上下页指针——4B**

记录

- 记录头——4B
- 数 据——18B

记录数量、校验和、上下页面指针	记录删除状态位向量 (111111.....00000000)

$$8+\frac{x}{8}+22x=4096$$

x=184

最简单的方式：页面头后，每22B存储一条记录。



页面组织——定长记录删除（一）

记录数量、校验和、上下页面指针				记录删除状态位向量 (11011100……000000)	
记录1	**记录2**	记录3	**记录4**	**记录5**	**记录6**
记录7	记录8	记录9	记录10	记录11	记录12
记录13	记录14	记录15	记录16	记录17	记录18

序号	SID	CID	Degree
1	2023011789	CS101	98
2	2023011689	CS101	83
3	2023011689	CS102	93
4	2023011189	CS101	98
5	2023011289	CS101	83
6	2023011389	CS102	93

页面组织——定长记录插入（一）

记录数量、校验和、上下页面指针				记录删除状态位向量 (1111100000100000)	
记录1	**记录2**	记录3**记录7**	**记录4**	**记录5**	**记录6**
记录7	记录8	记录9	记录10	记录11	记录12
记录13	记录14	记录15	记录16	记录17	记录18

序号	SID	CID	Degree
1	2023011789	CS101	98
2	2023011689	CS101	83
3	2023011689	CS102	93
4	2023011189	CS101	98
5	2023011289	CS101	83
6	2023011389	CS102	93
7	2023011389	CS101	93

最坏情况扫描整个磁盘块（遍历记录删除状态位向量），才能找到记录存放的位置。

页面组织——定长记录删除（二）

记录数量、校验和、上下页面指针						空闲链	7	记录删除状态位向量 (1111110000000100000)			
记录1		记录2		记录3		记录4		记录5		记录6	
记录7	8	记录8	9	记录9	10	记录10	11	记录11	12	记录12	13
记录13	14	记录14	15	记录15	16	记录16	17	记录17	18	记录18	nil

序号	SID	CID	Degree
1	2023011789	CS101	98
2	2023011689	CS101	83
3	2023011689	CS102	9?
4	2023011189	CS101	98
5	2023011289	CS101	83
6	2023011389	CS102	93

头插法，指针不占位置

记录数量、校验和、上下页面指针						空闲链	3	记录删除状态位向量 (1101111000000100000)			
记录1		记录2		记录3	7	记录4		记录5		记录6	
记录7	8	记录8	9	记录9	10	记录10	11	记录11	12	记录12	13
记录13	14	记录14	15	记录15	16	记录16	17	记录17	18	记录18	nil

页面组织——定长记录插入（二）

记录数量、校验和、上下页面指针				空闲链	3	记录删除状态位向量(1101111000000100000)	
记录1		记录2		记录3	7	记录4	记录5	记录6
记录7	8	记录8	9	记录9	10	记录10 11	记录11 12	记录12 13
记录13	14	记录14	15	记录15	16	记录16 17	记录17 18	记录18 nil

序号	SID	CID	Degree
1	2023011789	CS101	98
2	2023011689	CS101	83
3	2023011689	CS102	93
4	2023011189	CS101	98
5	2023011289	CS101	83
6	2023011389	CS102	93
7	2023011389	CS101	93

删除队头

记录数量、校验和、上下页面指针				空闲链	7	记录删除状态位向量(1111111000000100000)	
记录1		记录2		记录7		记录4	记录5	记录6
记录7	8	记录8	9	记录9	10	记录10 11	记录11 12	记录12 13
记录13	14	记录14	15	记录15	16	记录16 17	记录17 18	记录18 nil

页面组织——定长记录修改

记录数量、校验和、上下页面指针				记录删除状态位向量(11111100……000000)	
记录1	记录2	记录3	记录4	记录5	记录6
记录7	记录8	记录9	记录10	记录11	记录12
记录13	记录14	记录15	记录16	记录17	记录18

序号	SID	CID	Degree
1	2023011789	CS101	98
2	2023011689	CS101	83
3	2023011689	CS102	93
4	2023011189	CS101	98
5	2023011289	CS101	83
6	2023011389	CS102	93

93→95

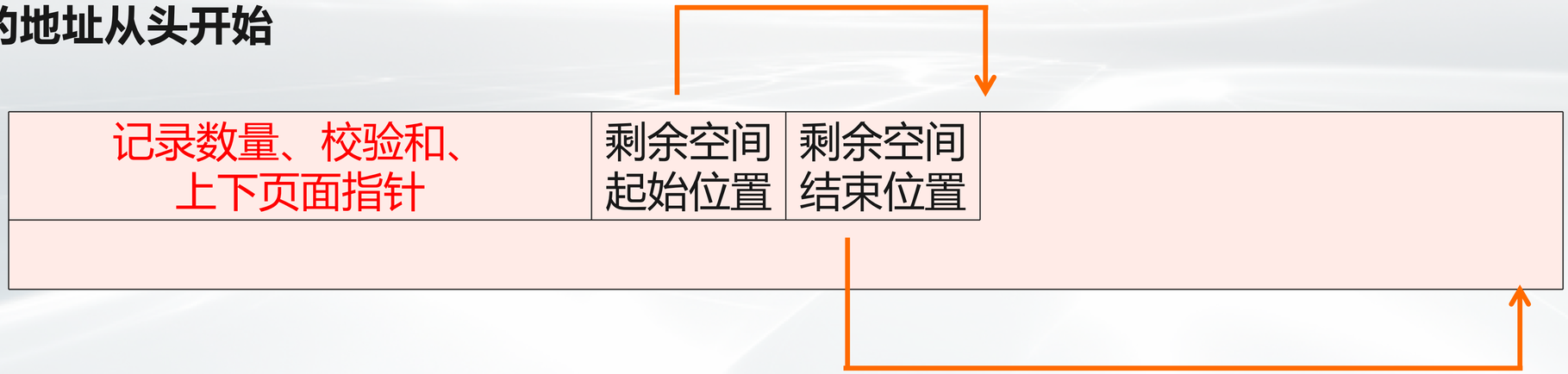
页面组织——变长记录

磁盘块结构

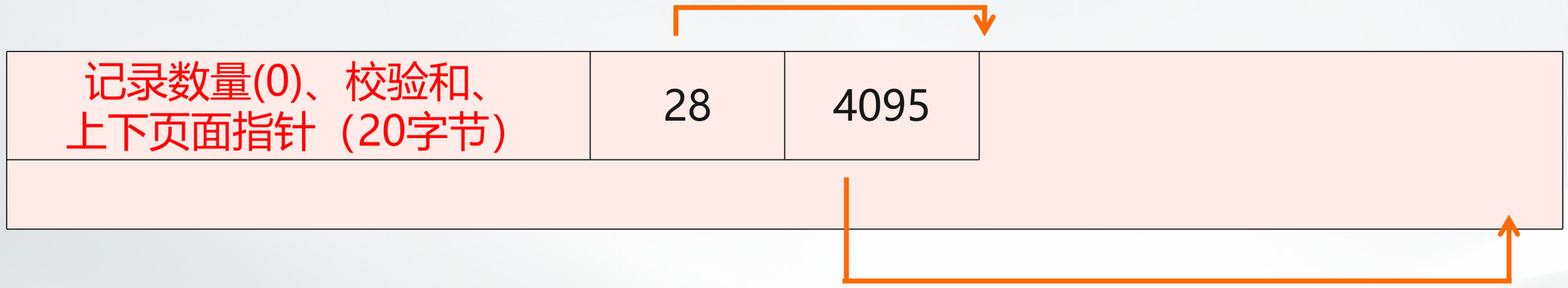
- 页面头：20B
- 槽：偏移量（2B）+长度（2B）
- 剩余空间指针：起始位置（4B）、结束位置（4B）。

存储策略——两头向中间逼近

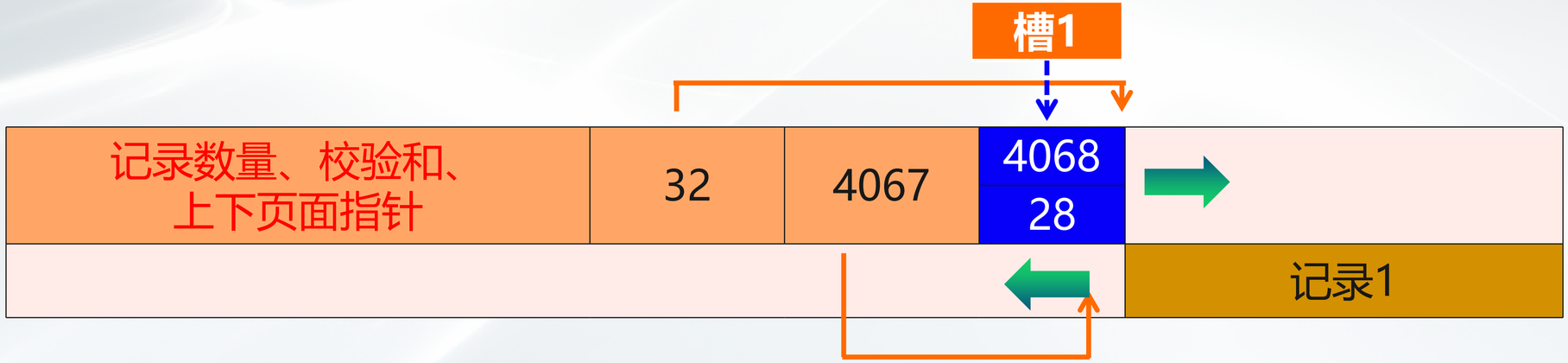
- 数据从磁盘块最后开始存，
- 记录的地址从头开始



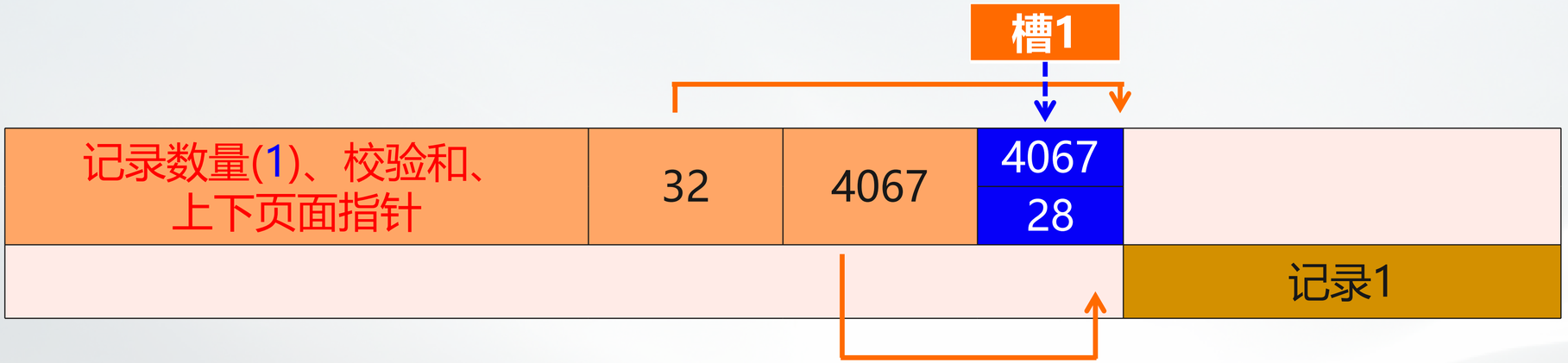
页面组织——变长记录插入



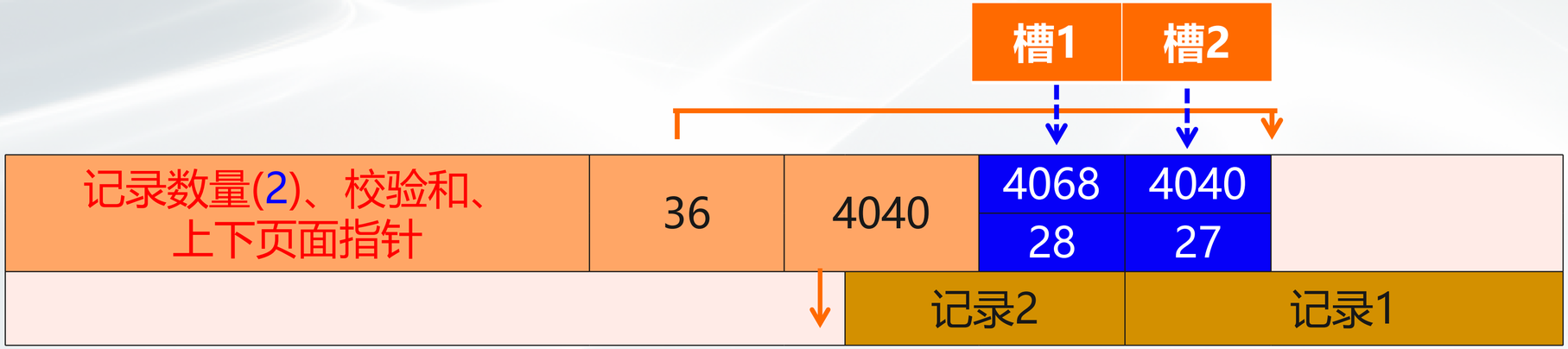
序号	SID	Name	GPA	Dept
1	2023011789	Wu Jun	3.9	计算机
2	2023011689	Li Bing	3.78	数学



页面组织——变长记录插入

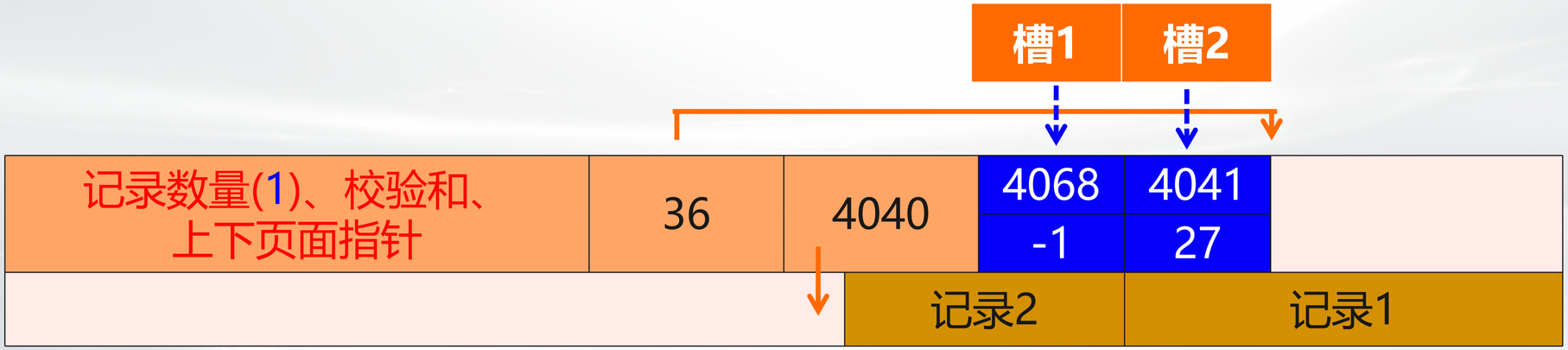


序号	SID	Name	GPA	Dept
1	2023011789	Wu Jun	3.9	计算机
2	2023011689	Li Bing	3.78	数学



页面组织——变长记录删除

序号	SID	Name	GPA	Dept
1	2023011789	Wu Jun	3.9	计算机 ✖
2	2023011689	Li Bing	3.78	数学



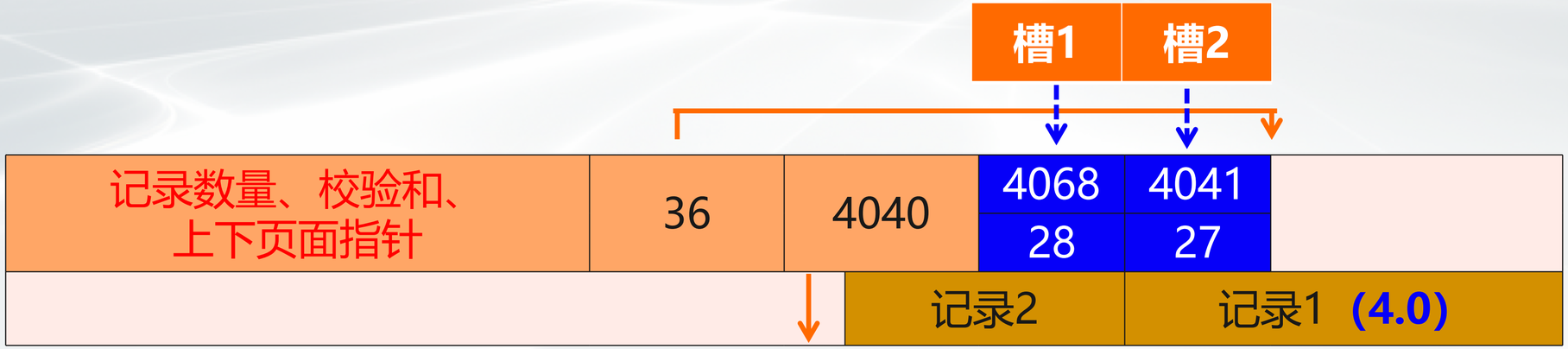
根据剩余空间的起始位置可以计算出槽的数目，并不一定等于有效的记录数。

页面组织——变长记录修改（一）

- 定长部分直接修改
- 变长部分
  - 修改后增加长度，删除+插入
  - 修改后长度变小，原地修改



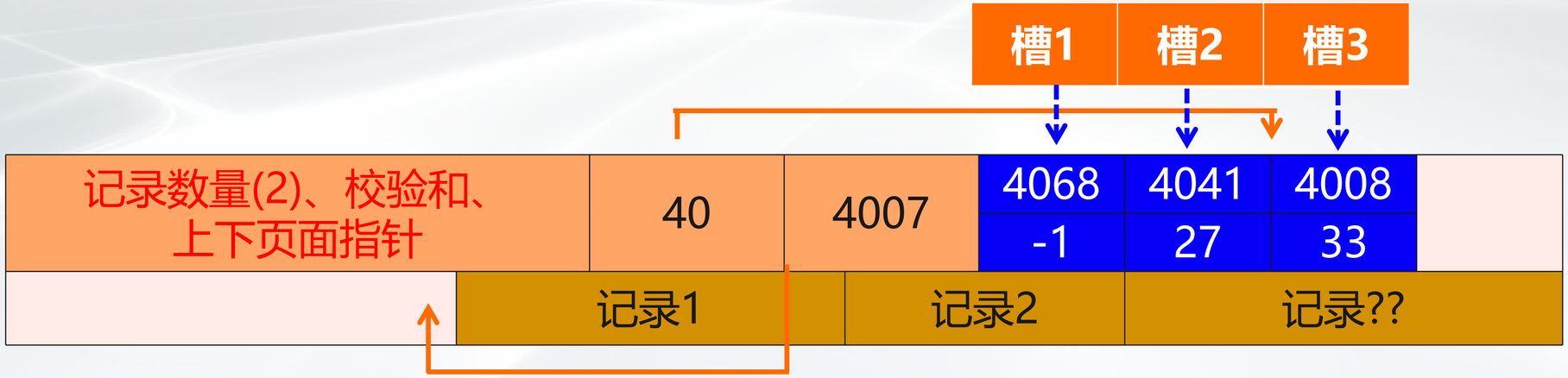
序号	SID	Name	GPA	Dept
1	2023011789	Wu Jun	3.9	计算机
2	2023011689	Li Bing	3.78	数学



页面组织——变长记录修改（二）

- 定长部分直接修改
- 变长部分
  - 修改后增加长度，删除+插入
  - 修改后长度变小，原地修改

序号	SID	Name	GPA	Dept
1	2023011789	Wu Jun **Rong**	3.9	计算机
2	2023011689	Li Bing	3.78	数学

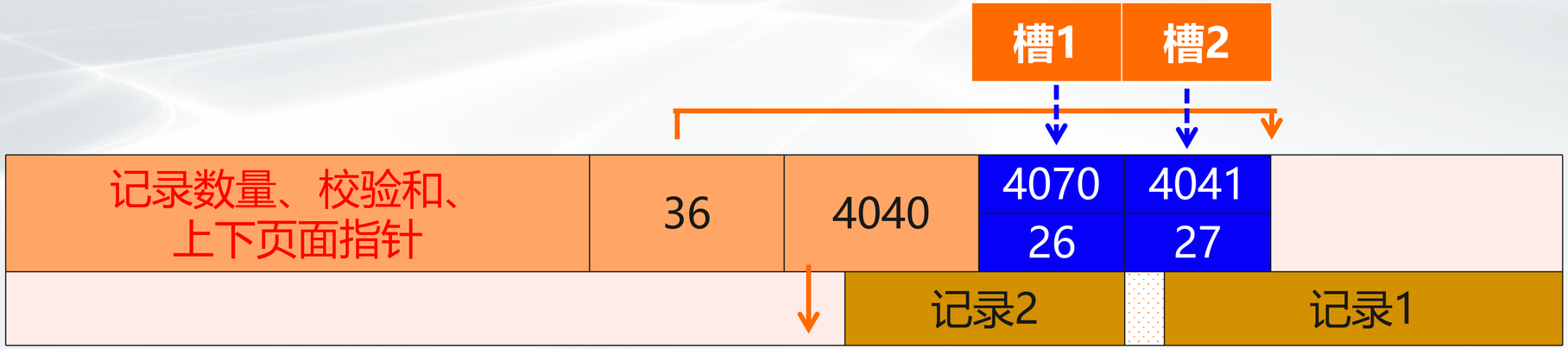


页面组织——变长记录修改（三）

- 定长部分直接修改
- 变长部分
  - 修改后增加长度，删除+插入
  - 修改后长度变小，原地修改

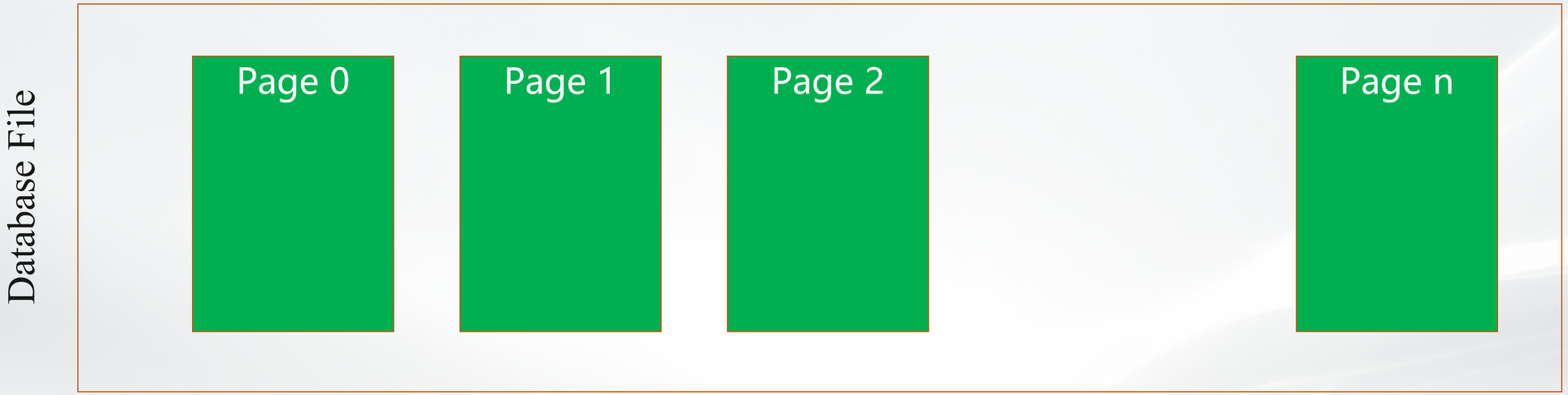
序号	SID	Name	GPA	Dept
1	2023011789	Wu Jun	3.9	计算机
2	2023011689	Li Bing	3.78	数学

外语

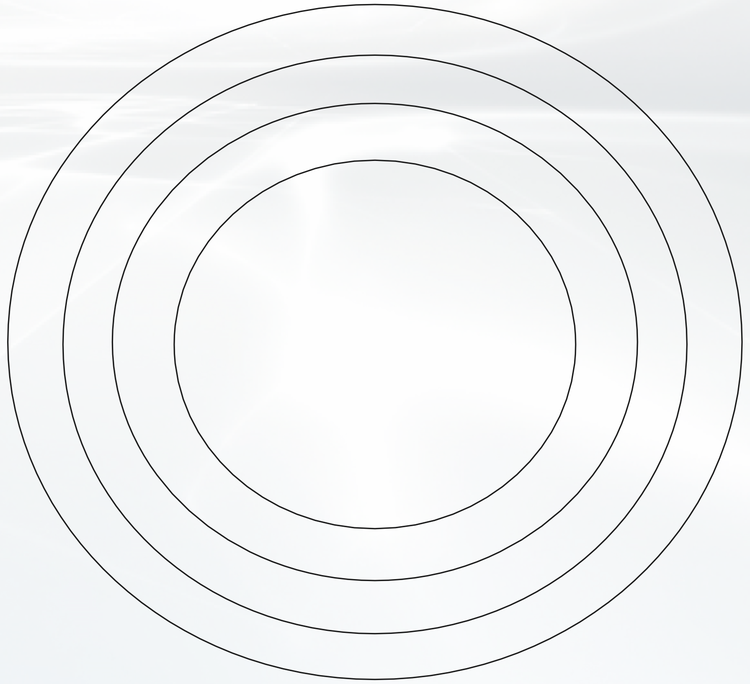


文件组织

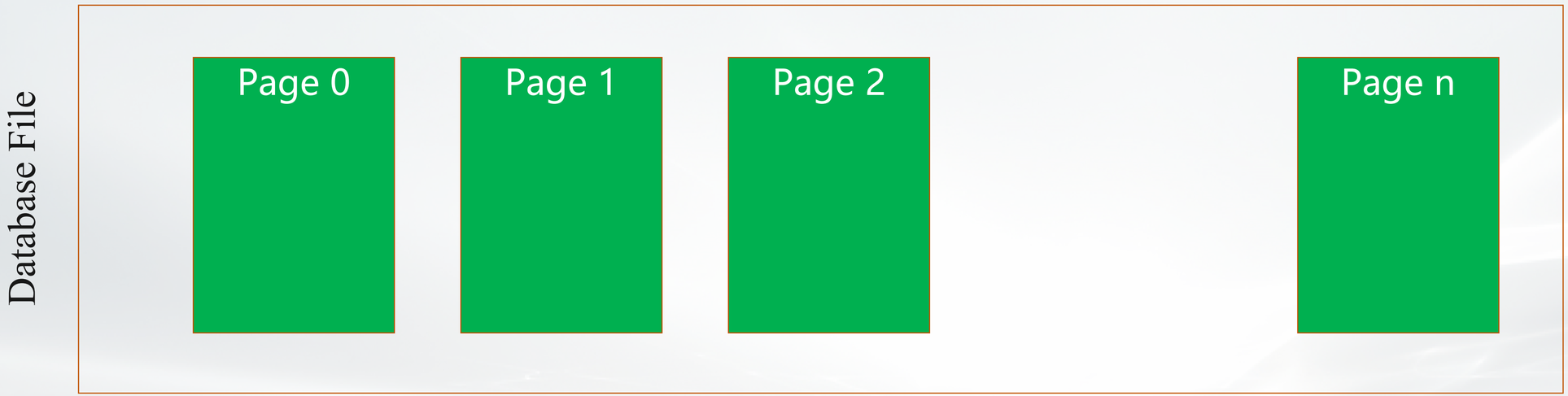
- 堆表
- 顺序表
- 哈希
- B+树
- 聚簇



- 物理上磁盘块顺序存放
- 磁盘块之间的逻辑关系（下一个磁盘块）
- 磁盘内的数据

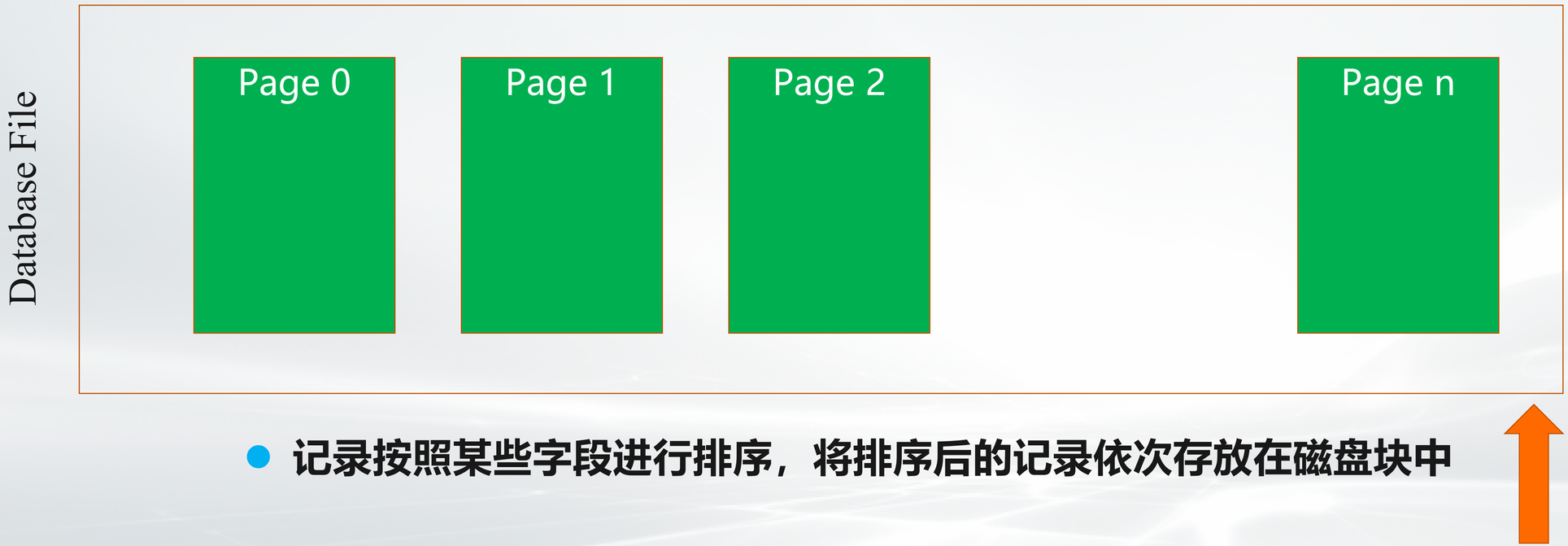


堆表（一）



- 逻辑上的下一块，就是物理上分配的下一块
- 插入记录比较好，当前存储块满了，存储到下一个磁盘块。
- 需要暴力扫描每一个磁盘块
- 数据库中创建关系时，虽然关系没有记录也会分配存储空间。

顺序表（一）



- 记录按照某些字段进行排序，将排序后的记录依次存放在磁盘块中

SID	Name	GPA	Dept
2023011789	Wu Jun	3.9	计算机
2023011689	Di Bing	3.78	数学
2023011781	Wu Kun	3.69	计算机
2023011682	Li Bing	4.02	数学
2023011769	Wu Hong	3.79	计算机

GPA

SID	Name	GPA	Dept
2023011682	Li Bing	4.02	数学
2023011789	Wu Jun	3.9	计算机
2023011769	Wu Hong	3.79	计算机
2023011689	Di Bing	3.78	数学
2023011781	Wu Kun	3.69	计算机

顺序表（二）

Database File

Page 0

SID	Name	GPA	Dept	Next
2023011789	Wu Jun	3.9	计算机	2:0
2023011689	Di Bing	3.78	数学	1:0

Page 1

SID	Name	GPA	Dept	Next
2023011781	Wu Kun	3.69	计算机	nil
2023011682	Li Bing	4.02	数学	0:0

Page 2

SID	Name	GPA	Dept	Next
2023011769	Wu Hong	3.79	计算机	0:1

- **频繁的插入与删除，需要移动记录代价大**
- **物理不变，利用链表存放排序结果**
- **指针：块号—+块内地址**

- **每一块不装满，留出一定的空白去给将来插入**
- **装填因子**

SID	Name	GPA	Dept
2023011789	Wu Jun	3.9	计算机
2023011689	Di Bing	3.78	数学
2023011781	Wu Kun	3.69	计算机
2023011682	Li Bing	4.02	数学
2023011769	Wu Hong	3.79	计算机

GPA

序号	SID	Name	GPA	Dept	
1	2023011789	Wu Jun	3.9	计算机	5
2	2023011689	Di Bing	3.78	数学	3
3	2023011781	Wu Kun	3.69	计算机	nil
4	2023011682	Li Bing	4.02	数学	1
5	2023011769	Wu Hong	3.79	计算机	2

顺序表（三）

Database File

Page 0

SID	Name	GPA	Dept	Next
2023011789	Wu Jun	3.9	计算机	2:0
2023011689	Di Bing	3.78	数学	1:0

Page 1

SID	Name	GPA	Dept	Next
2023011781	Wu Kun	3.69	计算机	nil
2023011682	Li Bing	4.02	数学	2:2

Page 2

SID	Name	GPA	Dept	Next
2023011769	Wu Hong	3.79	计算机	0:1
2023011489	Lu Jun	3.93	计算机	0:0

- **频繁的插入与删除，需要移动记录代价大**
- **物理不变，利用链表存放排序结果**
- **指针：块号—+块内地址**

SID	Name	GPA	Dept
2023011489	Lu Jun	3.93	计算机

插入

序号	SID	Name	GPA	Dept	next
1	2023011789	Wu Jun	3.9	计算机	5
2	2023011689	Di Bing	3.78	数学	3
3	2023011781	Wu Kun	3.69	计算机	nil
4	2023011682	Li Bing	4.02	数学	1
5	2023011769	Wu Hong	3.79	计算机	2

哈希表

- 根据Hash值确定桶数
- 将磁盘块分配到每一个桶中
- 桶内顺序存放

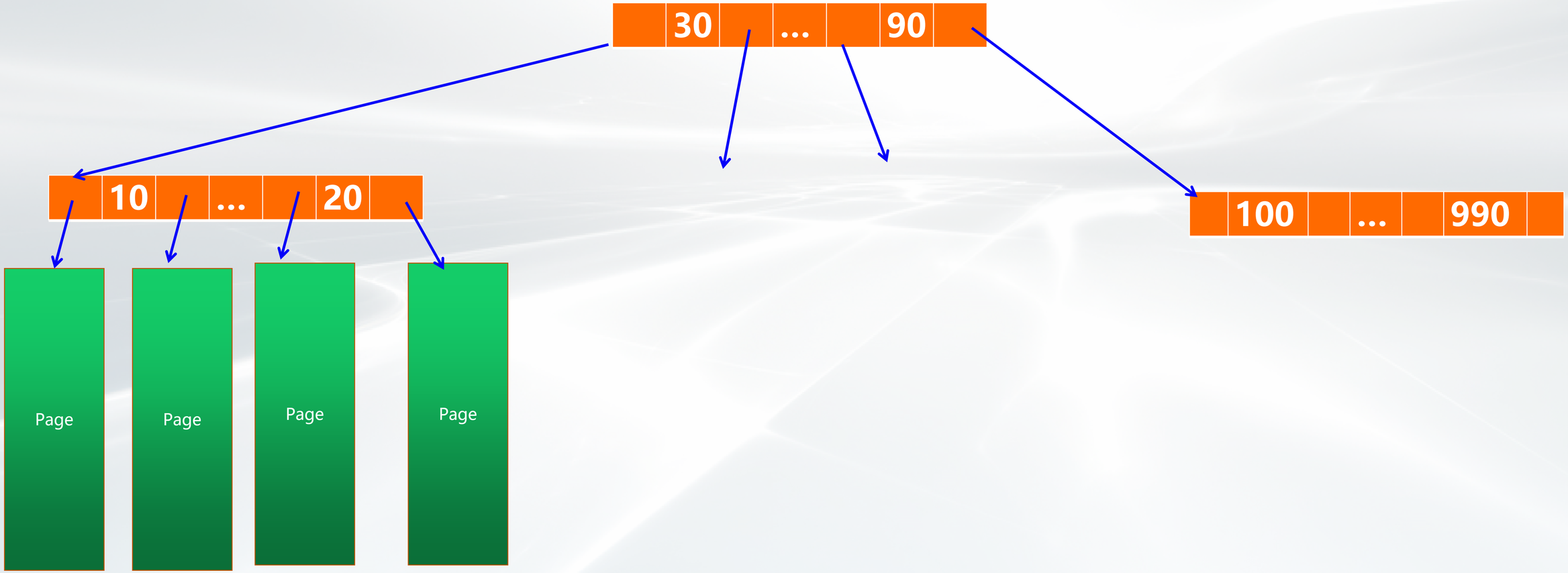


- 分成4个桶
- SID后4位相加，除4的余数
- 假设每个块存放2个元组

序号	SID	Name	GPA	Dept	位置
1	2023011789	Wu Jun	3.9	计算机	4:0
2	2023011689	Di Bing	3.78	数学	0:0
3	2023011781	Wu Kun	3.69	计算机	4:1
4	2023011682	Li Bing	4.02	数学	5:0
5	2023011769	Wu Hong	3.79	计算机	12:0

B+树(IOT)

- 叶子节点存储真实的记录
- 非叶子节点存放索引
- 适用于数据相对不变的情况（例如某个大学的学生基本信息），查询高效的情况。



B+树

- 假设一块是16K
- 身份证号码18位，18B，地址指针8B，一个索引对是26B，16KB/26B=630
- 每人的基本信息是100B，每个块可以存放160人
- B+树最多可以存放630*630*160人的信息，63,504,000.

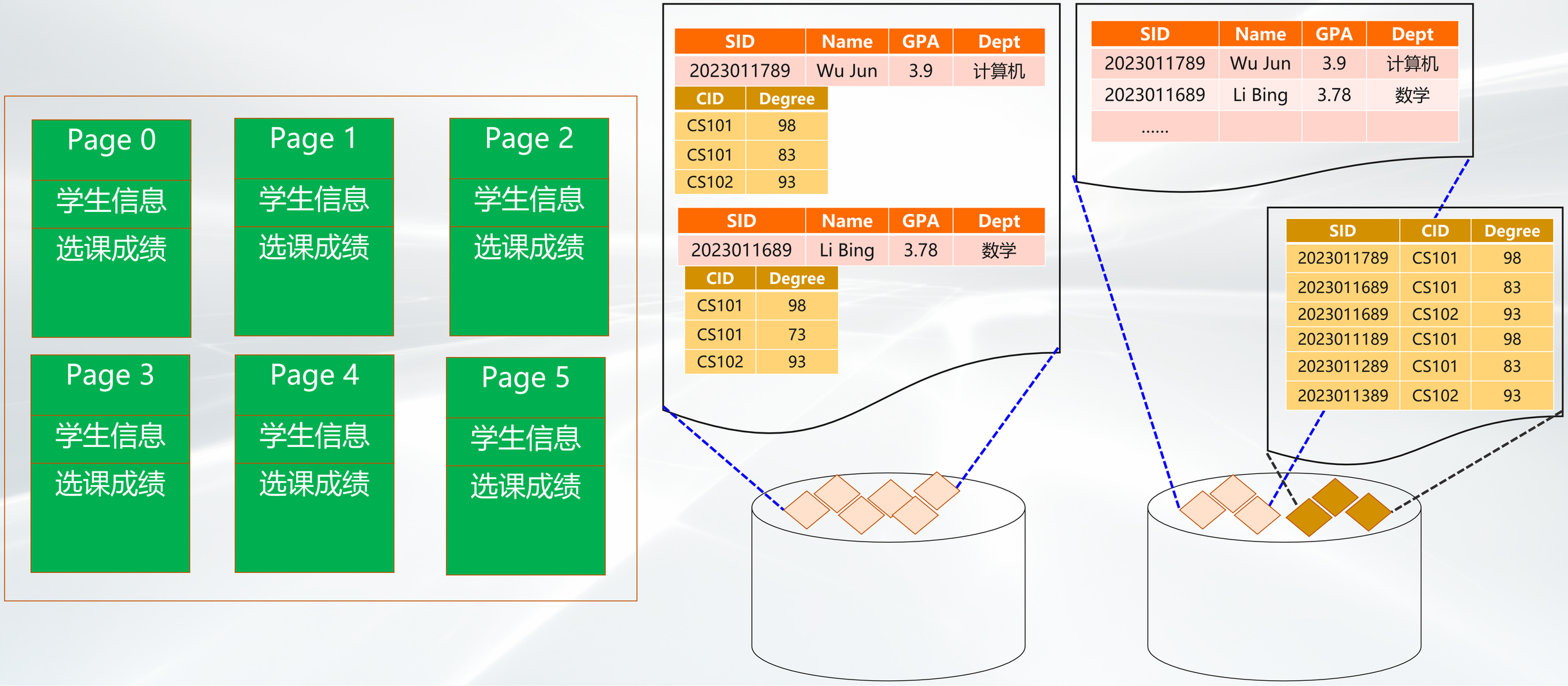
B+查询效率

- 如果B+树的根节点和中间节点在内存，需要的存储空间11M，访问数据，额外读一次硬盘
- 如果不用B+树，那么整个信息读入内存需要7G，此时才可以二分查找。
- 很好的平衡空间和时间的效率。

- 对查询性能有极致要求，三层全部放入内存。
- 磁盘块中头信息增加磁盘块的类别：索引块、数据块。
- 索引块和数据块在一个文件中。

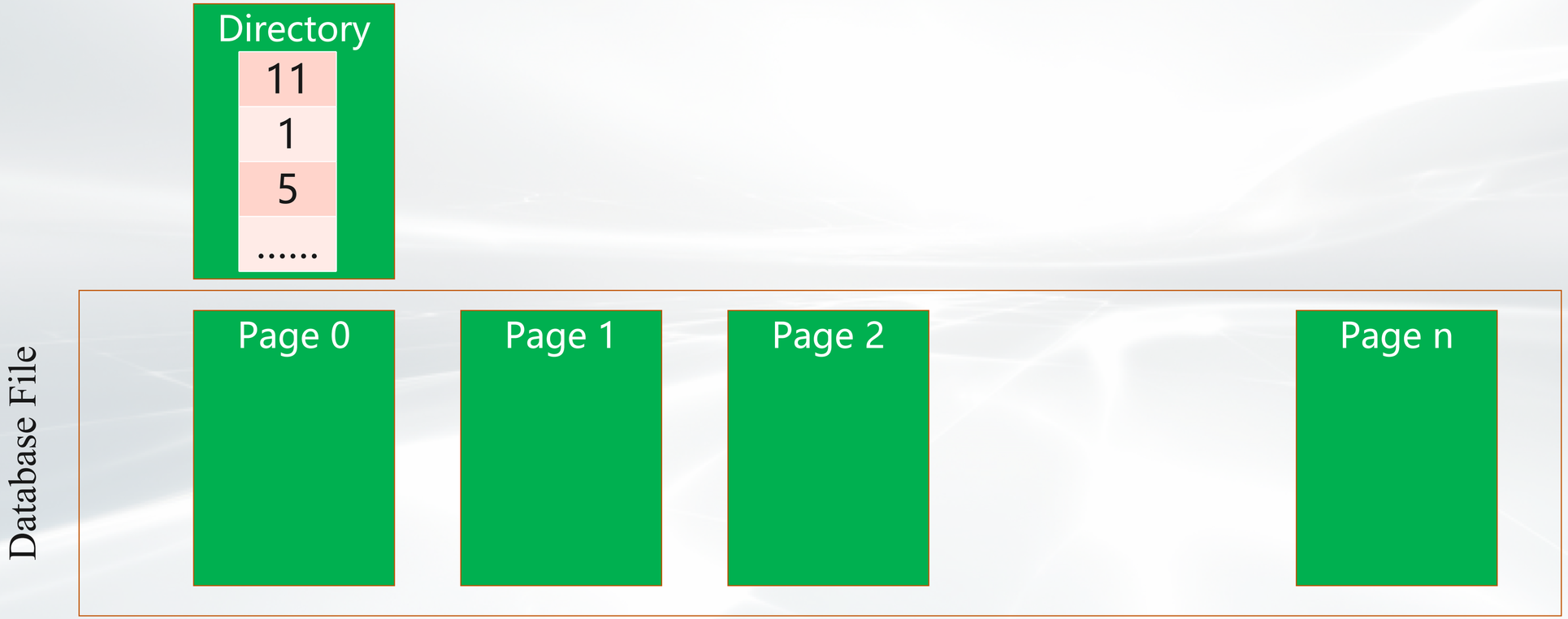
多表聚簇

- 一个磁盘块中存储多个关系的数据
- 适用于经常做链接的情况非叶子节点存放索引



空闲空间管理（一）

- 为每一个磁盘块建立空白空间目录，假设磁盘是4K
- 目录就是一个数组，第i个元素对应于第i块的空白空间情况
- 数组长度是1个字节，值等于1代表空白空间位于16B到31B之间



大小	15B	32B	47B			3700	4000B
换算	0	2	2			231	250
页面编号	1	2	3	4	5	6	7

空闲空间管理（二）

- 4KB用1B表示
- 关系是100M，那么目录大小是100*1024/4=25600B
- 扫描这个数组还是需要很长时间，建立二级空闲，N个连续页面为一组，存储最大值

空闲比例	233	131
页面编号	1-4	5-8

空闲比例	30	233	10	0	125	131	10	23
页面编号	1	2	3	4	5	6	7	8

- 用大根堆代替数组

记录插入算法

Insert into sc values(‘2023045123’,’CS123’78)

Algorithm： InsertTuple

输入：sql:
输出：SQLCODE

(1) tbl ← ParaseSQL(sql)
(2) fp ← FindFile(tbl)
(3) IF fp = NULL OpenFile(tbl)
(4) block_id← FindBlock()
(5) IF block_id= NULL
    AllocateBlock()
    UpdateDirectory()
    WriteTuple
  ELSE
    buf_id ←FindMemory(block_id)
    IF buf_id = NULL ReadBlock(block_id)
    UpdateDirectory()
    WriteTuple()
  ENDIF

记录删除算法

Algorithm： DeleteTuple

输入：sql:
输出：SQLCODE

(1) tbl ← ParaseSQL(sql)
(2) fp ← FindFile(tbl)
(3) IF fp = NULL OpenFile(tbl)
(4) block_id← FindBlock()
(5) buf_id ←FindMemory(block_id)
(6) IF buf_id = NULL ReadBlock(block_id)
(7) UpdateDirectory()
(8) DeleteTuple()

定长记录修改算法

Update ....

Algorithm： UpdateTuple

输入：sql:
输出：SQLCODE

```
(1) tbl ← ParaseSQL(sql)
(2) fp ← FindFile(tbl)
(3) IF fp = NULL OpenFile(tbl)
(4) WHILE blk ← NextBlock(fp)
        WHILE tup ← NextTuple(blk )
            IF tup 符合条件
                UpdateTuple();
                SetDirty()
            ENDIF
        END
    END
```

变长记录修改算法

Algorithm： UpdateTuple

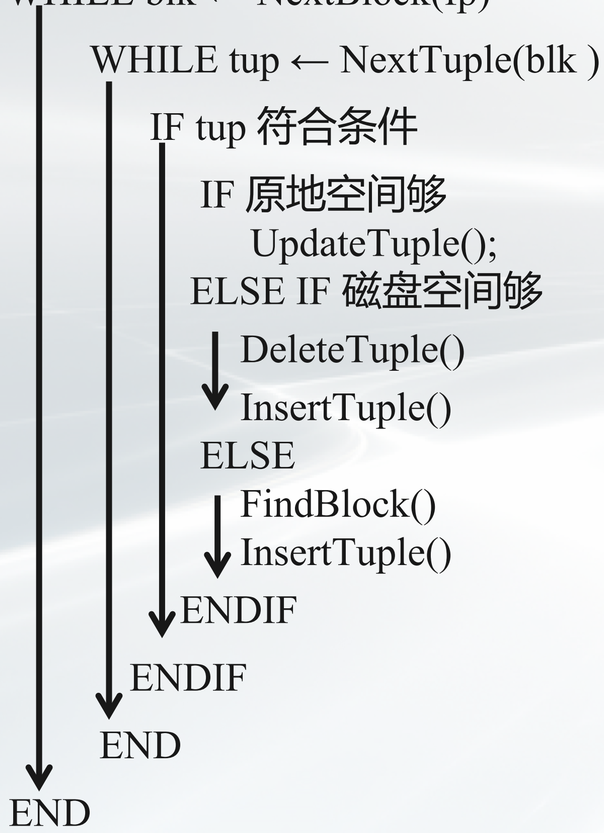
输入：sql:
输出：SQLCODE

(1) tbl ← ParaseSQL(sql)

(2) fp ← FindFile(tbl)

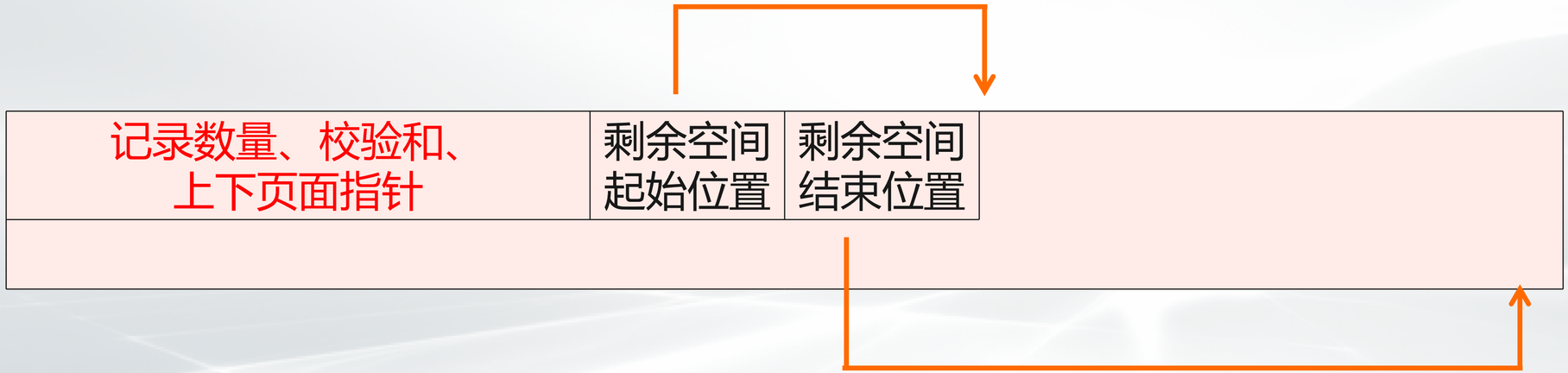
(3) IF fp = NULL OpenFile(tbl)

(4) WHILE blk ← NextBlock(fp)



跨块存储（一）

- 一个磁盘块存不下，需要存储多个磁盘块
- 磁盘块中的数据之间必须存储双向指针链。
- 磁盘块的类别：普通数据块、**扩展第一块、扩展块**，

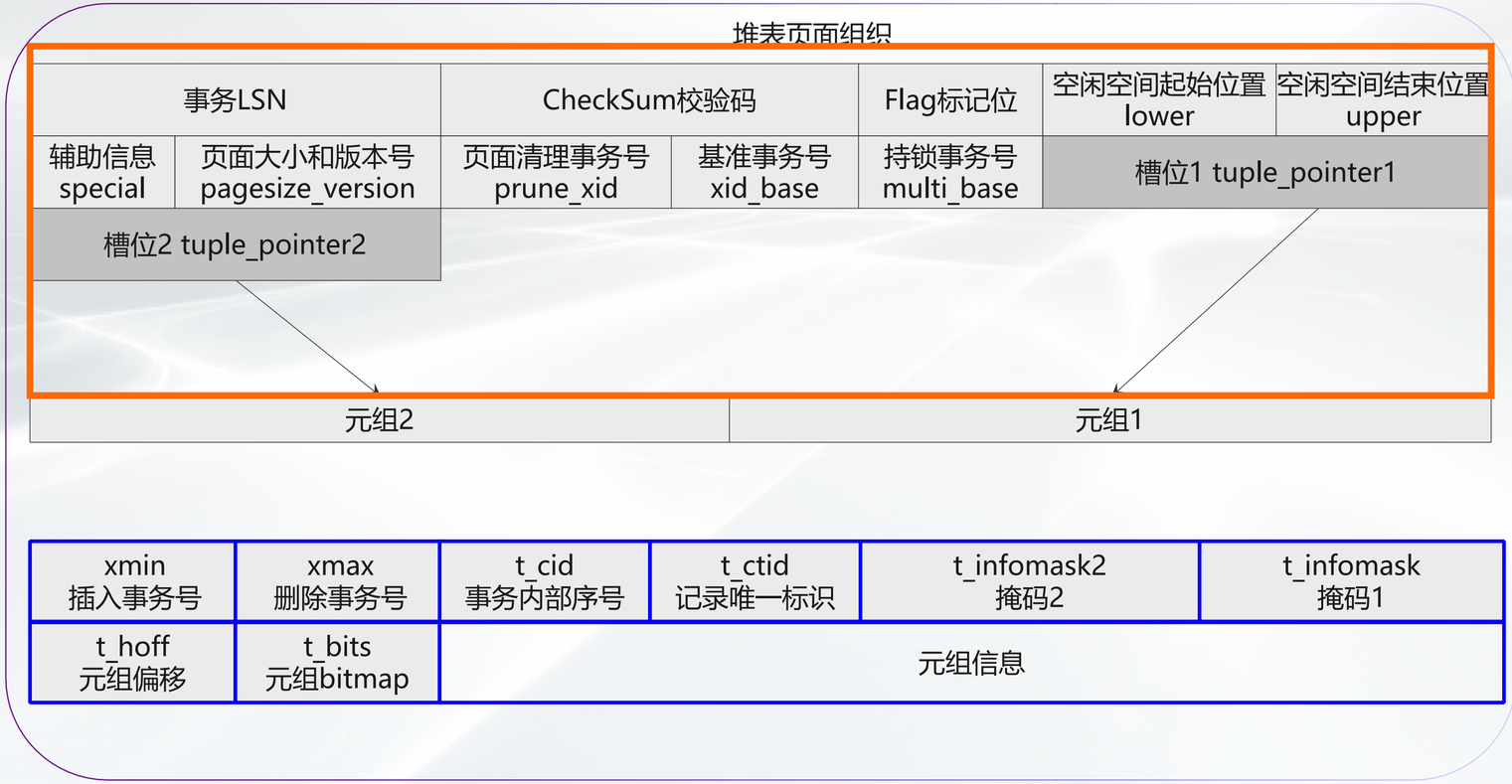


跨块存储（二）


长度
前驱
后继
块号4
记录数量、校验和、
上下页面指针<3,8>
类块别
1
-1
-1
32
-1
12
4064
-1
20
长度
前驱
后继
块号12
记录数量、校验和、
上下页面指针<3,8>
类块别
2
-1
-1
32
4
21
4064
20
20
长度
前驱
后继
块号21
记录数量、校验和、
上下页面指针<3,8>
类块别
2
32
3995
3996
12
-1
100
20
-1

GaussDB堆文件存储引擎—页面组织(一)

- 行存储以页面为单位
  - 页头（HeapPageHeader）记录页面的公用信息以及关键标识；
  - 页头后放置指向元组的行指针（tuple_pointer），并向页面尾部扩展。
  - 行指针指向的元组（tuple），从页面尾部向页面头部延展

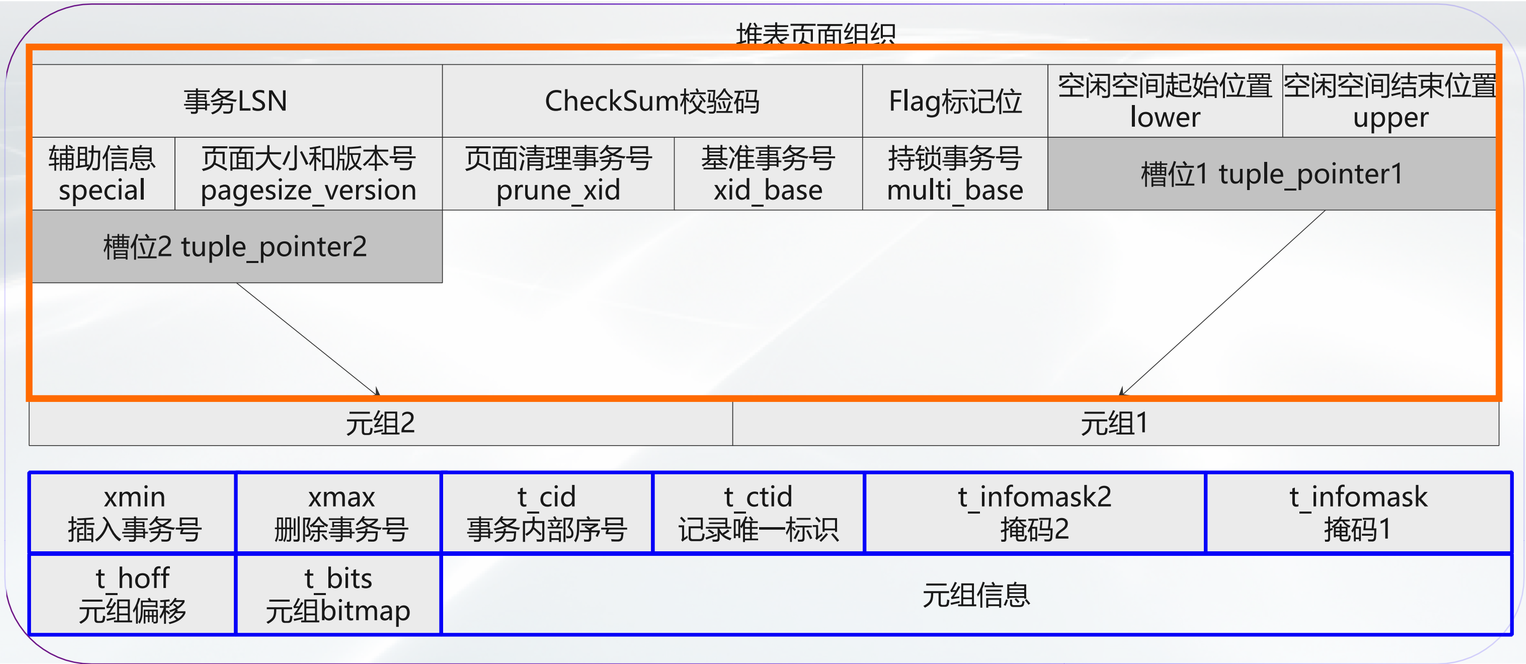


GaussDB堆文件存储引擎—页面组织(二)

标识	含义
pd_lsn	事务LSN,该页面最后一次修改操作的预写日志结束位置，用于检查点推进和保持恢复操作的幂等性
pd_checksum	校验和即页面的CRC校验值
pd_flag	页面标记位，用于保存各类页面相关的辅助信息:空闲的元组指针、页面是否已满、页面元组是否都可见、页面是否被压缩、页面是否是批量导入的、页面是否加密、页面采用的CRC校验算法
pd_lower	页面中间空闲空间的起始位置，即下一个可以插入元组的起始位置
pd_upper	页面中间空闲空间的结束位置，即当前已使用的元组指针数组的尾部。
pd_special	辅助信息，页面尾部特殊区域的起始位置。
pd_pagesize_version	页面大小和版本号
pd_prune_xid	页面清理辅助事务号(32位),通常为该页面内现存最老的删除或更新操作的事务号
pd_xid_base	该页面内所有元组的基准事务号(64位)
pd_multi_base	持锁事务号
tuple_pointer	槽位指针，即元组指针，指向页面中具体的元组

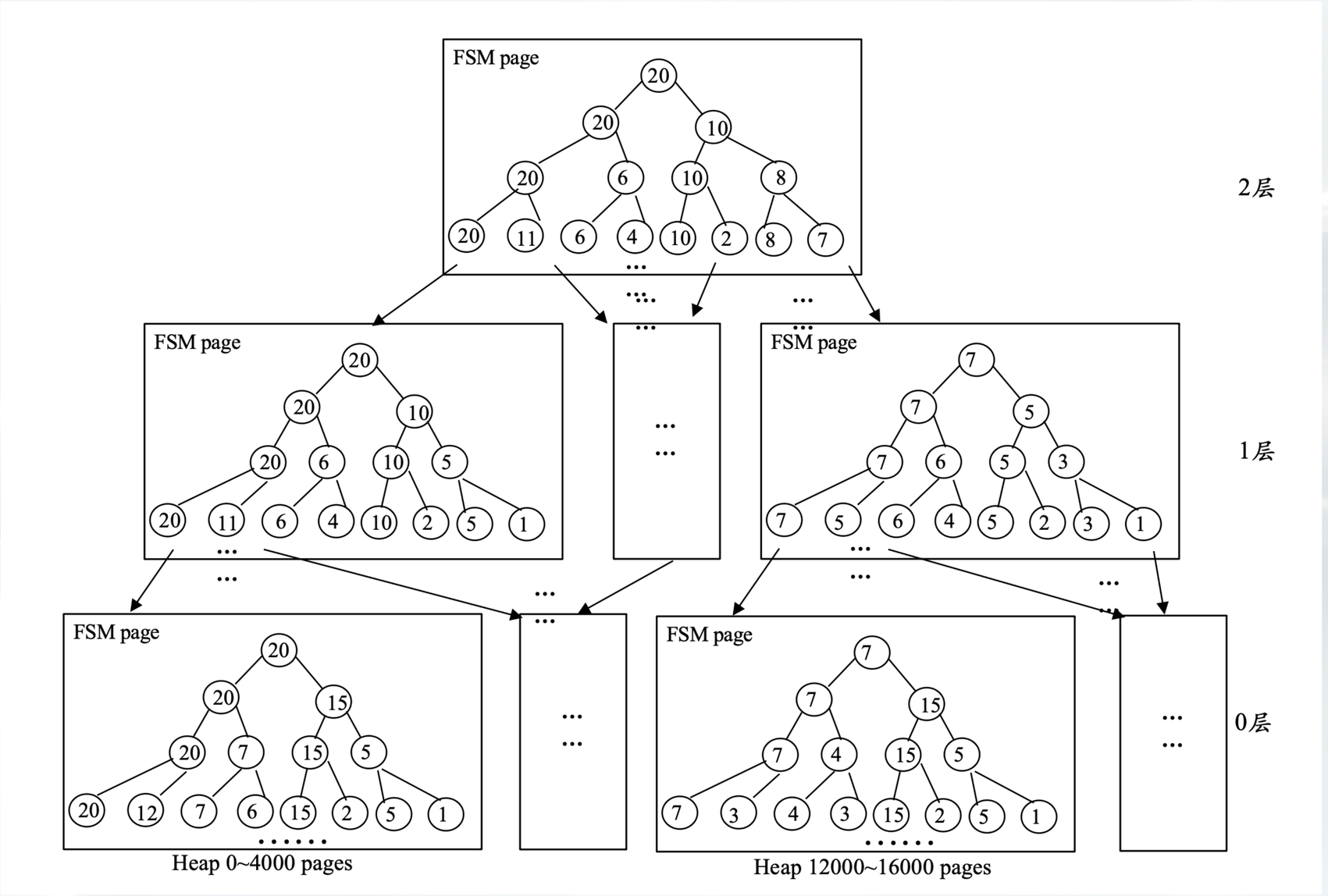
GaussDB堆文件存储引擎—元组组织（一）

元组标识	含义
xmin	插入元组的事务号XID(32位)
xmax	删除或更新元组的事务号(32位)。如果元组还没有被删除，则为零
t_cid	对该记录的操作在事务内部的序号
t_ctid	当前元组的页面和页面内元组指针下标
t_infomask2	元组属性掩码，包含元组中字段个数、HOT更新标记、HOT元组标记等
t_infomask	元组另一个属性掩码，包含是否有空字段标记、是否有变长字段标记等信息
t_hoff	元组数据距离元组头部结构体起始位置的偏移。
t_bits	所有字段的NULL值bitmap。每个字段对应t_bits中的一个bit位。



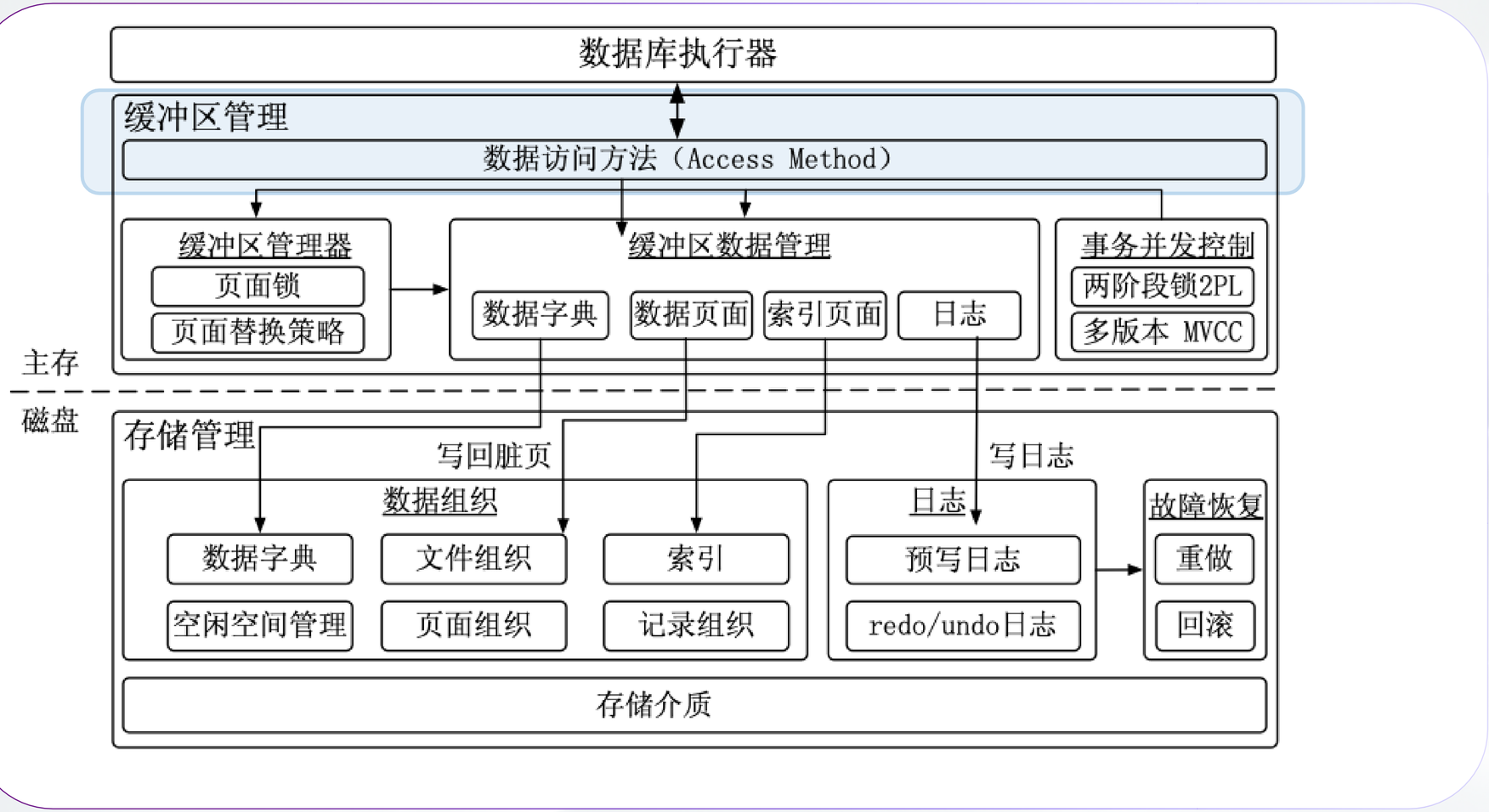
空闲空间管理——GaussDB

- 页面8KB
- FSP页面大小是8KB
- 每个页面内部采用大根堆的二叉树来管理空闲空间，4000个页面
- 总共4000^3



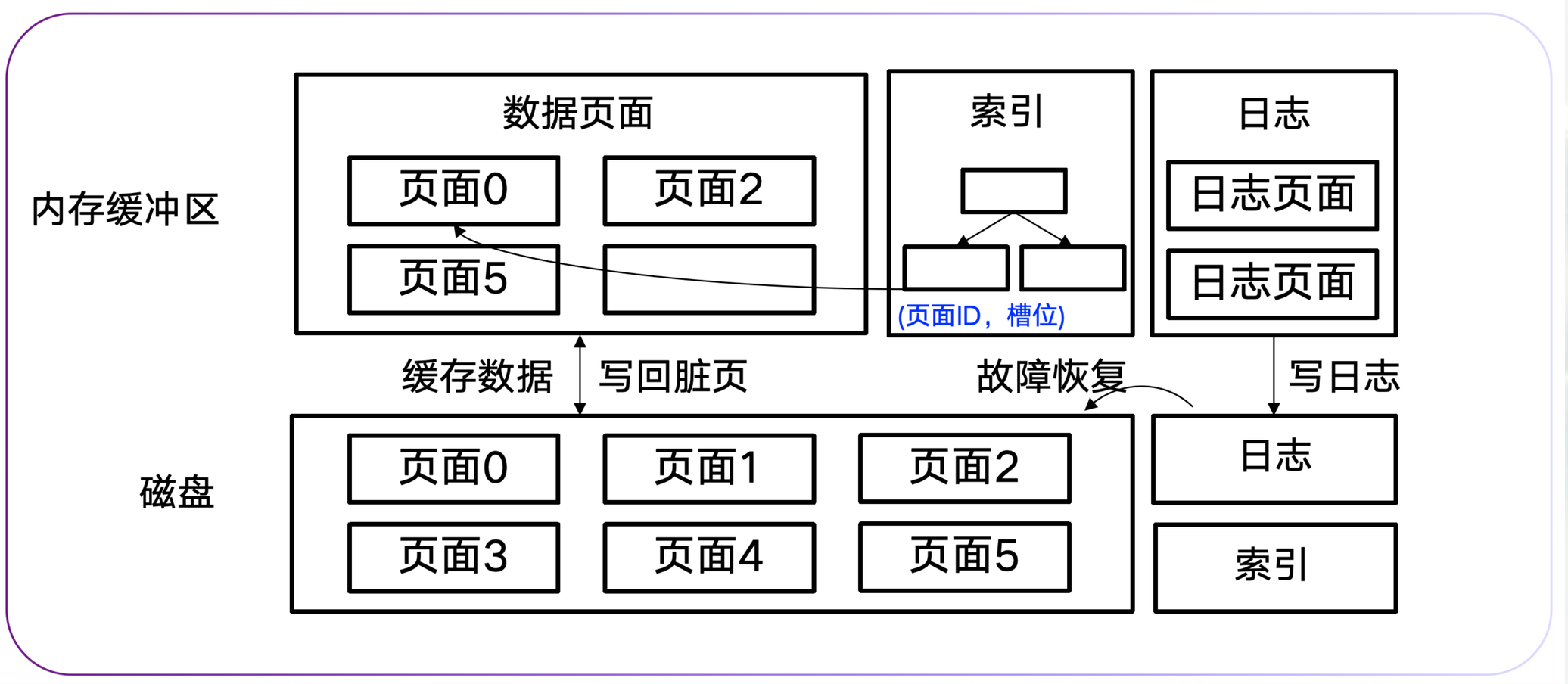
缓冲区管理

DBMS自己接管OS的功能——将存储于磁盘的数据同时存储于主存，减少磁盘访问



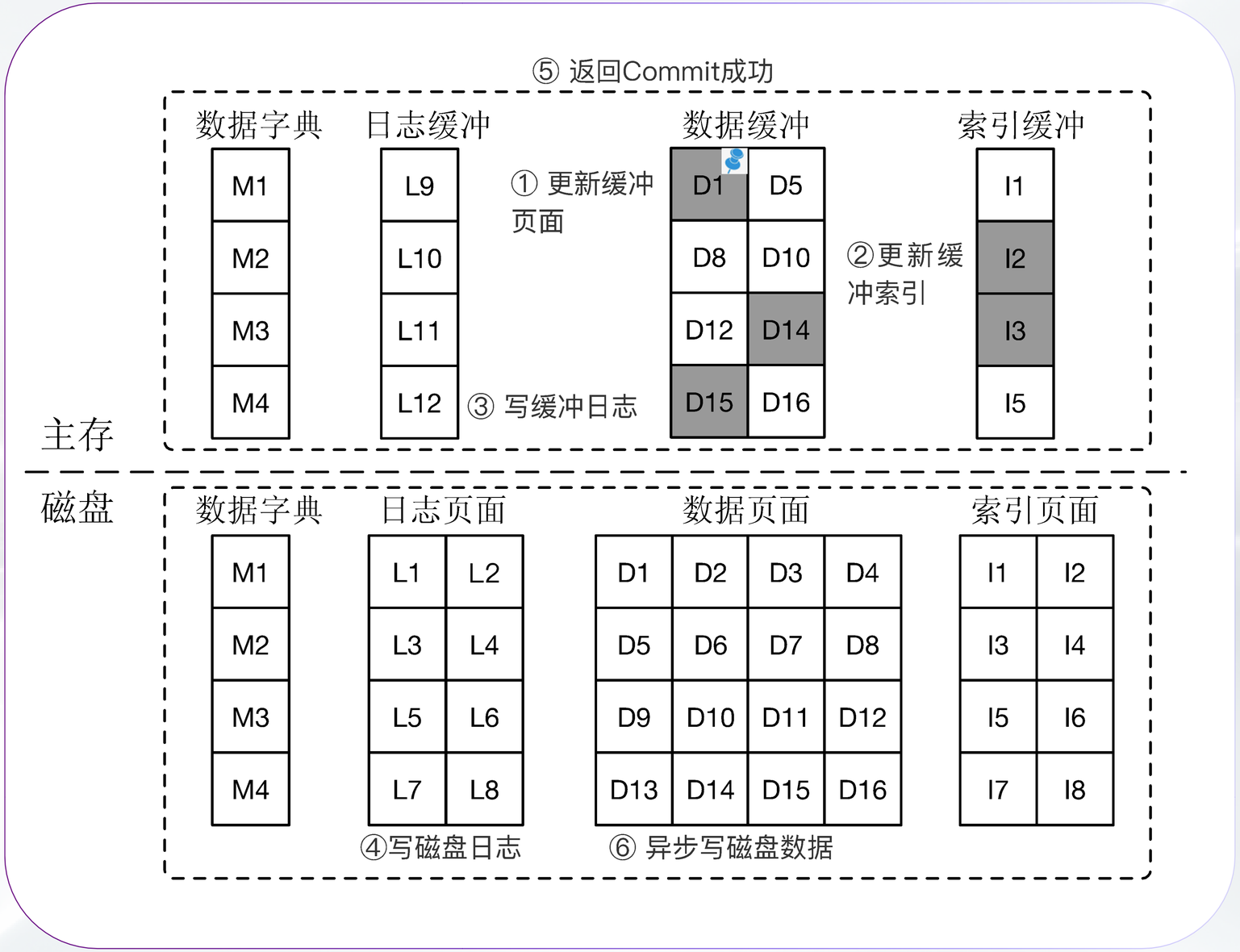
缓冲区管理

缓冲区数据主要包含缓存的磁盘数据页面、索引、日志等。日志可用于故障恢复。



缓冲区管理

1. 更新缓冲区页面
2. 更新缓冲区索引
3. 更新缓冲区日志
4. 更新磁盘日志
5. 返回提交成功
6. **异步刷新数据和索引页面**



缓冲区管理器

为数据访问提供读取、修改和写入接口

读页面时

1.如果该页面已经在缓冲区中，则返回缓冲区中的页面数据

2.如果该页面不在缓冲区中，则从磁盘上读取该页面

(1)如果缓冲区未满，则将数据写入缓冲区

(2)如果缓冲区已满，则根据缓冲区页面替换策略挑选其他页面剔除或写回到磁盘上，再读入新页面

写页面时

1:如果该页面在缓冲区中，则写入缓存、日志和索引

2:如果该页面不在缓冲区中，则从磁盘上读取该页面，再写入页面到缓存、日志和索引

缓冲区页面替换策略

- 最近最少使用策略LRU
  - 前1次使用时间越晚（越近期）的页面将来越有机会被使用，越不应该被替换
- 最不经常使用策略LFU
  - 使用频率越高的页面将来越有机会被使用，越不应该被替换
- LRU-K策略
  - 前第K次访问的时间越晚的页面将来越有机会被使用，越不应该被替换

参考书

- 数据库系统实现
- 数据库系统概念（第七版）
- 数据库系统基础（第七版）
- 数据库管理系统——从基本原理到系统构建
- 数据库管理系统原理与实现



